



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

4D1 - Information Management

SQL Project

The Implementation of a Mii data management database for the Nintendo Wii

-By Stephen Komolafe (21336975)



Contents

Description of Database

- 1. Domain Description
- 2. Entity Relationship Diagram for the Database
- 3. Mapping ER Diagram to Relational Schema.....

Explanations of data and SQL Code used to implement database:

- 4. Explanation and SQL Code for Creating the database Tables (including constraints).....
- 5. Explanation and SQL Code for Populating the Tables.....
- 6. Explanation and SQL Code for Querying the database.....

1. Domain Description

Originally created on the Nintendo Wii, Miis are customisable avatars introduced by Nintendo, designed for players to express themselves digitally within various games and applications. These characters feature a range of customisable attributes—from physical appearance, such as hair and facial features, to more personal information like nicknames and birthdates. Miis became a central part of the user experience, allowing players to carry personalised avatars across games and share between consoles through the Wiis online functionality.

Despite their presence on more recent consoles like the Nintendo Switch, the database I've created specifically focuses on managing Mii data on the Nintendo Wii platform. With this domain, I aim to express the relationships between Mii data and the players they represent and/or were created by, the software they are used in, and the consoles they were stored on. The entities chosen to convey this domain are as follows:

➤ Mii

The Mii entity contains attributes that can be used to easily identify any one specific Mii, such as personal information or more immediately visible physical traits like head type, skin tone, and hair type.

- **Mii Attributes:**

- Mii ID (Primary Key) (Required)

- Each Mii receives a unique identifier known as a Mii ID upon creation. When a Mii is used in software, rather than creating a complete copy of the Miis data to write save data to, the Mii ID is used to save data to the corresponding Mii directly [1]. Due to its importance in the console/software-wide management of Mii data, the Mii ID was chosen as the primary key in the Mii entity.

Other attributes in the Mii entity include:

Attribute	Description
Nickname	Stores the name of the Mii as a varchar(15 variable), a Nickname is required to create a Mii thus it cannot be NULL
Birthday	An optional date field representing the Mii's birthday. If not specified by the player, it defaults to NULL.
Favorite Colour	Stores the Mii's favourite shirt colour as a varchar(2) variable. It uses a colour code system (e.g., R for Red, O for Orange). As this field cannot be NULL it defaults to a value of 'R' (Red) if unspecified.
Mii Creator	An optional varchar(15) variable to store the name or alias of the player who created the Mii. It defaults to NULL if not provided.
Mingle	A tinyint variable with a default value of 0. It indicates whether the Mii can be shared with other consoles (1 for "Yes," 0 for "No").
Favorite?	A tinyint variable with a default value of 0. It marks a Mii as a "favorite" if set to 1, which affects sorting and visual appearance (change in pants colour).
Gender	A single-character varchar(1) variable that denotes the gender of the Mii. It can be either 'M' for male or 'F' for female. It cannot be NULL as it is the first option asked of the player to choose when creating a Mii.
SkinTone*	A varchar(3) variable that determines a Mii's skin tone. Cannot be NULL so it defaults to '1,1'.
HeadType*	A varchar(3) variable that determines a Mii's head shape. Cannot be NULL so it defaults to '1,1'.

HairType**	A varchar(5) variable that determines the Mii's hair style. Cannot be NULL - Miis can be bald, however this is treated the same as other hair styles ("3,4,1"). Hair defaults to '1,1,1'.
Hair Colour^*	A varchar(3) variable that determines hair colour. Employs a (row),(column) format to accommodate the lookup from pre-defined grids of attributes. [Explanation of format found in Appendix] Cannot be NULL so it defaults to '1,2'.
Height & Weight	Int variables, both defaulting to 5. Each uses a scale from 1 to 10, representing different ranges for Mii height and body type. Can be used to dictate in-game character traits [i.e. Weight Class in Mario Kart Wii]. Cannot be Null
PlayNum	An int that acts as a foreign key linking to the player table, representing the player who created or last used the Mii. Cannot be NULL.
ConID	A foreign key linking to the console table. Is used to track the console where the Mii was created or last used. Cannot be NULL.
MiiChanID	A foreign key linking to the software table. Is used to track the specific channel used to create the Mii. Cannot be NULL.
^ Colours available are vague/subject to opinion therefore uses grid system instead of colour code system * Uses a (row),(column) format to accommodate the lookup from pre-defined grids of attributes. ** Uses a (page),(row),(column) format to accommodate the lookup from pre-defined grids of attributes. [Visual guide of grid format found in Appendices]	

➤ Facial_attributes

As there are a myriad of components that make up the appearance of a Mii, there was no feasible way to condense them all into the Mii entity. For this reason, a 'facial_attributes' weak entity dependent on the 'Mii' entity was created for the express purpose of capturing the detailed characteristics that make up a Miis face.

These attributes are:

Attribute	Description
FacialFeatures*	A varchar(3) variable to represent optional features like freckles, wrinkles, or makeup. Can be set to NULL
EyeType**	A varchar(5) variable to define the eye shape/type. Defaults to '1,1,1'.
EyeColour*	A varchar(3) variable to define the eye colour. Defaults to '1,1'.
EyeBrowType**	A varchar(5) variable to define the eyebrow style/type. Defaults to '1,1,1'.
EyeBrowColour^*	A varchar(3) variable that defines eyebrow colour. Defaults to '1,2'.
NoseType*	A varchar(3) variable to define the nose type. Cannot be NULL, so defaults to '1,1'.
MouthType**	A varchar(5) variable to define the mouth type. Cannot be NULL, so defaults to '1,1,1'.
GlassesType*	A varchar(3) variable that holds the type of glasses (if any) the Mii wears Allows NULL to indicate no glasses.
GlassesColour	A varchar(2) variable for glasses colour. It uses a colour code system (e.g., 'BK' for Black, 'BR' for Brown). Cannot be NULL so defaults to 'BK'.

FacialHairType*	A varchar(15) variable that holds beard and mustache data in the format [Beard(B)](row),(column) [Mustache(M)](row),(column). Allows for NULL values.
FacialHairColour^*	A varchar(3) variable for facial hair colour Defaults to a value of '1,1'.
Mole	A tinyint(1) variable indicating whether the Mii has a mole (1 for yes, 0 for no). Is set to 0 by default.
miID	Is an int variable that links each record in facial_attributes to a specific Mii, using the MiID from the mii table as a foreign key.
^ Colours available are vague/subject to opinion therefore uses grid system instead of colour code system * Uses a (row),(column) format to accommodate the lookup from pre-defined grids of attributes. ** Uses a (page),(row),(column) format to accommodate the lookup from pre-defined grids of attributes. [Visual guide of grid format found in Appendices]	

➤ Player

The Player entity refers to every individual who has created and/or used a Mii. By linking players to Miis, the database can track which player created or last used a specific Mii, enabling personalised experiences and accurate tracking of each player's interactions across different games and applications.

- **Player Attributes:**

- PlayerNo (Primary Key)

Once a player's Wii Remote is paired with the console, they are assigned a number from 1 to 4, based on the connection order. Since the Wii predates Nintendo's email-based accounts, these player numbers are the console's only way of tracking connected users. Thus, PlayerNo serves as the primary key for the Player entity.

The only other attribute present in the Player entity is:

Attribute	Description
RepsMID	A foreign key, which is used to link the player to the specific Mii that represents them during gameplay. Cannot be NULL.

➤ Console

The console entity represents the Wii system where all data management is taking place, acting as the central device through which players access and manage Miis and software. This entity is necessary for associating each Mii and software title with a specific hardware unit.

- **Console Attributes:**

- Console ID (Primary Key)

Each console is assigned a unique ID upon manufacture in the format of
[XXXX-XXXX-XXXX-XXXX] (e.g. 1234-5678-9012-3456)

Which serves as the hardware identifier within the Wii's managing data ecosystem. This ConsoleID enables the system to manage console-specific data, including associated software and Miis, making it an ideal primary key for the Console entity.

Other attributes in the Console entity include:

Attribute	Description
Storage(MB)	A varchar(12) variable which records the console's current internal storage capacity, specifically for storing channels, WiiWare games, etc. Must have a value for each console; it cannot be NULL.

FirmwareVersion	A varchar(4) variable to store the firmware version of the console in the format "X.X(region)". Where the region code indicates the console's specific firmware (e.g., U for USA, E for Europe, J for Japan). Firmware version is required for every console entry; it cannot be NULL.
OnlineID	A varchar(4) variable that holds a unique ID for the online functionality of each console. In the format of [YYY-YYY-YYY-YYY] (e.g., 123-456-789-012). Each console must have a unique OnlineID; it cannot be NULL

➤ Software

The software entity represents the various disc-based games, downloadable titles (WiiWare), and other channels available on the Nintendo Wii. The software entity helps track the different titles that are compatible with Mii data [2], linking them to the console to ensure access to such data.

- **Software Attributes:**

- *TitleID* (Primary Key)

Every piece of Wii software, be it disc or channel, is distinguished by its own unique identifier known as a TitleID, making it an effective primary key for the software entity. GameTDB (gametdb.com) was used in the sourcing of each Wii software's TitleID.

Other attributes include:

Attribute	Description
Type	A varchar(45) variable that specifies the type of the software; this is also indicated by the character length of its TitleID (6 for disc IDs, 4 for downloadable title/stock channel IDs).
Title	A varchar(45) variable which stores the name of the software
Genre	A varchar(15) variable to define the genre of the software [e.g., action, sports, etc.]
Developer	A varchar(45) variable which stores the name of the developer responsible for creating the software.
Publisher	A varchar(45) variable that stores the name of the company responsible for distributing the software.
ReleaseDate	Stores when the software was made publicly available in the YYYY-MM-DD format.
FileSize(GB)	A double variable that shows the storage space required by the software.
Region	A varchar(6) variable that defines the geographical distribution area of the software ['NTSC-J' for Japan, NTSC-U for Americas, PAL for Europe, etc.]
HrdwID	The foreign key, which refers to ConsoleID in the console table.
Are all important in the identification of software - None of these attributes can be NULL.	

2. Entity Relationship Diagram for the Database

With each entity explained, the following diagrams allow for a visual depiction of relationships within the complete database:

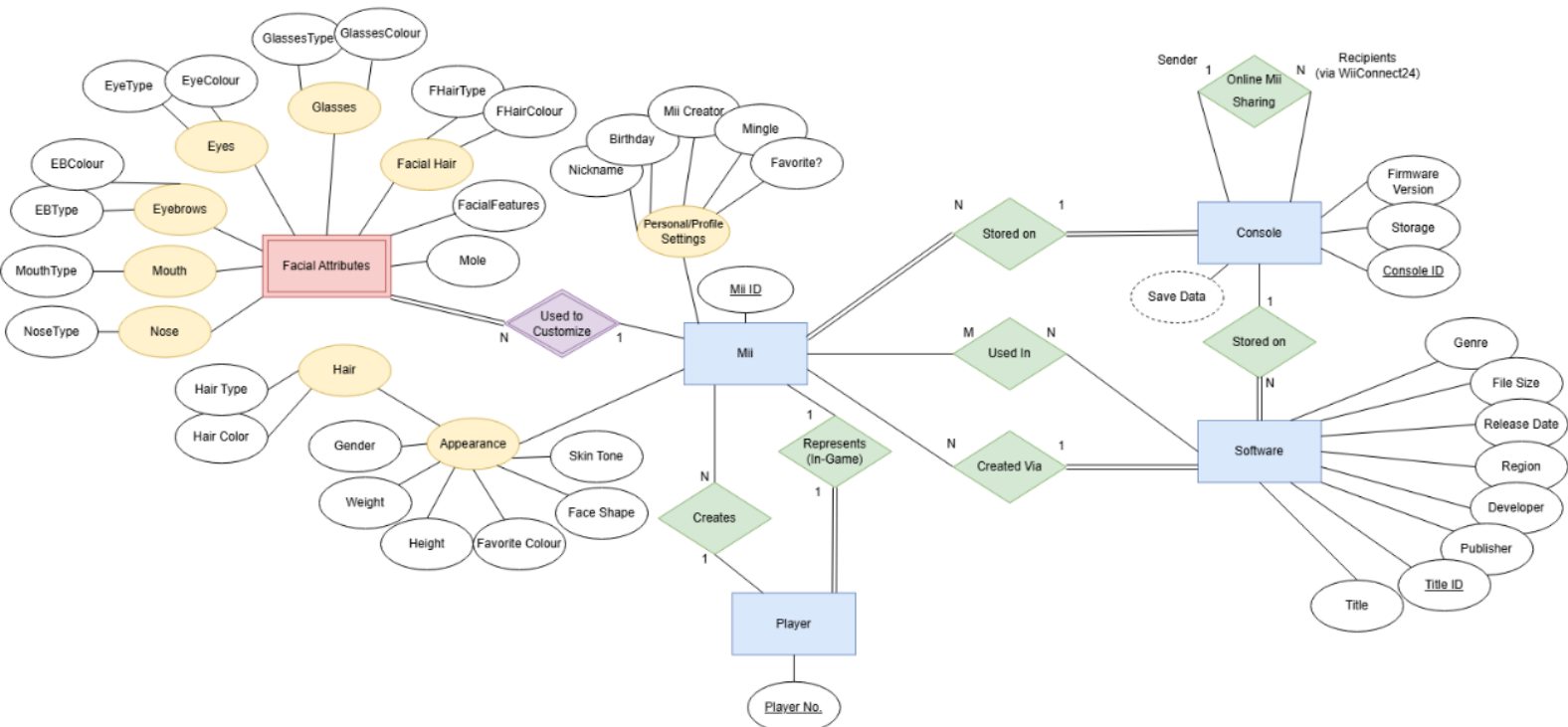


Figure 1a. ER-Diagram(Chen Notation)

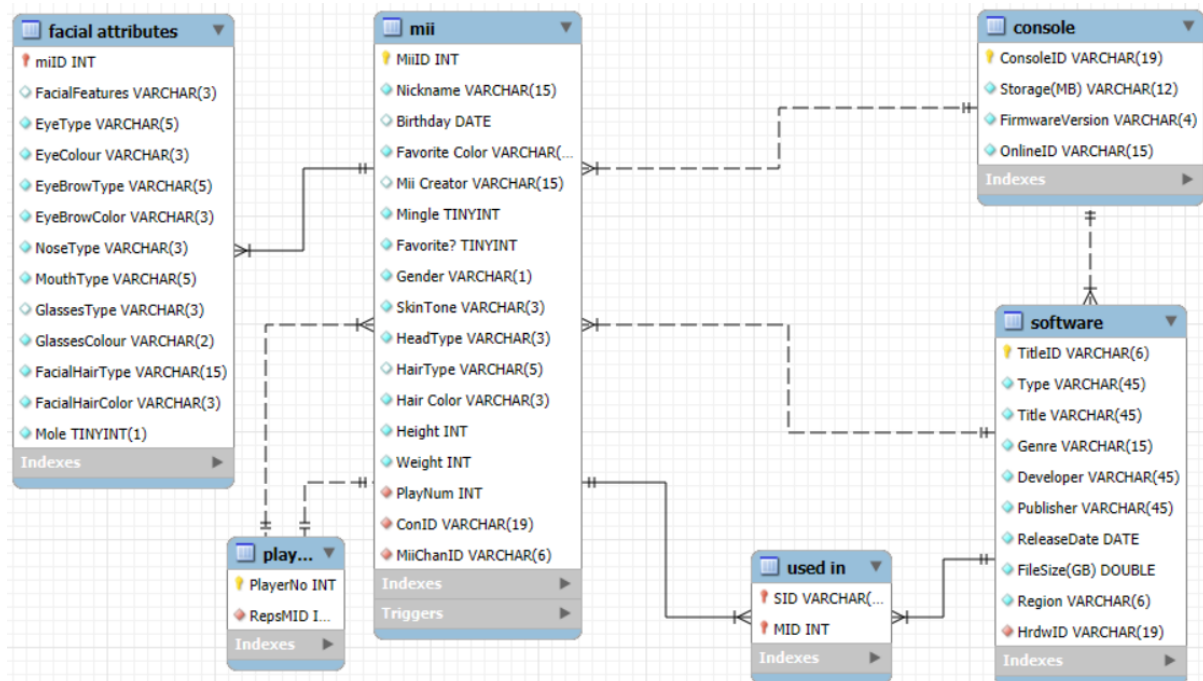


Figure 1b. ER-Diagram(Chen Notation)

3. Mapping ER Diagram to Relational Schema

Using the 7-step algorithm covered in lectures prior to this assignment, the ER diagram could be mapped to a relational schema.

Step 1: Mapping of Regular Entity Types

Mii, Player, Console, and Software relations were created in the schema in reference to the regular entities in the ER diagram, and Mii ID, PlayerNo., ConsoleID, and TitleID were defined as their respective primary keys indicated by the underline on each.

Mii													
<u>Mii ID</u>	Nickname	Birthday	FavColor	MiiCreator	Mingle	Favourite?	SkinTone	HeadType	HairType	HairColor	Height	Weight	Gender

Player	
<u>PlayerNo.</u>	

Console		
<u>ConsoleID</u>	Storage	FirmwareV

Software								
<u>TitleID</u>	Type	Title	Genre	Developer	Publisher	RlsDate	FileSize	Region

Figure 2a. Step 1 in mapping relational schema

Step 2: Mapping of Weak Entity Types

A Facial Attributes relation was created to reflect the weak entity of the same name. The primary key Mii ID of the Mii relation is used as a foreign key attribute in this relation, now referred to as miiID.

Mii													
<u>Mii ID</u>	Nickname	Birthday	FavColor	MiiCreator	Mingle	Favourite?	SkinTone	HeadType	HairType	HairColor	Height	Weight	Gender

Facial Attributes												
<u>miiID</u>	FacialFeat.	EyeType	EyeColor	EBType	EBColor	NoseType	MouthType	GlsType	GlsColor	FHairType	FHairColor	Mole

Player	
<u>PlayerNo.</u>	

Console		
<u>ConsoleID</u>	Storage	FirmwareV

Software								
<u>TitleID</u>	Type	Title	Genre	Developer	Publisher	RlsDate	FileSize	Region

Figure 2b. Step 2 in mapping relational schema

Step 3: Mapping of Binary 1:1 Relation Types

As only one Mii can represent each player during gameplay, the entities share a 1:1 relationship. To indicate this in the schema, a RepsMID attribute was created and associated with the Player relation, as it has total participation in the relationship.

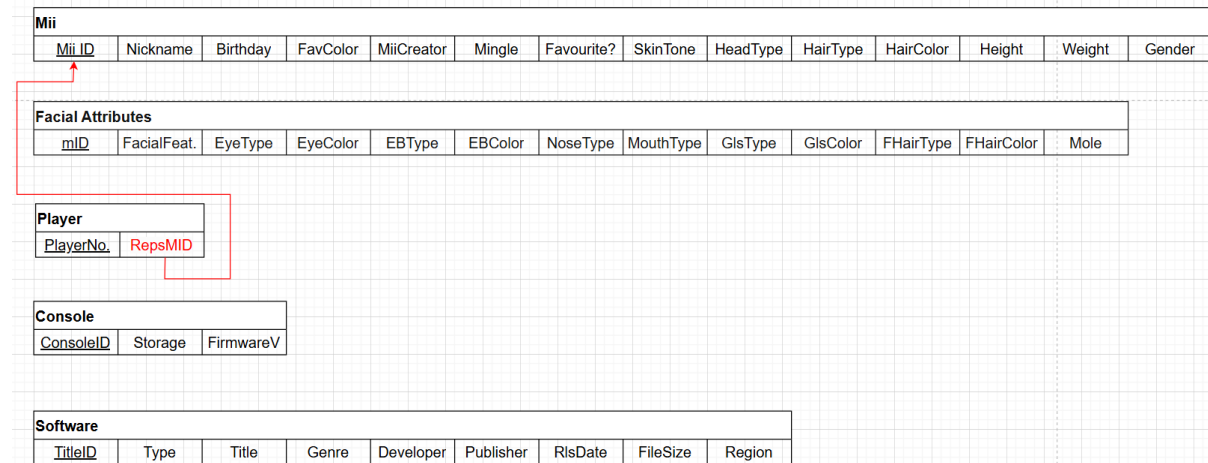


Figure 2c. Step 3 in mapping relational schema

Step 4: Mapping of Binary 1:N Relationship Types

- Many Mii's can be stored on a single console, created by one player, and created using a single software. Taking this into account, the primary keys of Console, Player, and Software entities are included in the Mii relation as foreign keys as ConID, PlayNum, and MiiChanID, respectively.
- As one Mii can possess many facial attributes, the mID attribute of the relation can be used to depict this.
- Many software can be stored on a single console; thus, the primary key of Console is seen as a foreign key in the Software relation referred to as HrdwID.
- One console can receive Mii data from many other consoles via online connectivity. Because of this, a recursive relationship is present in the Console entity. To show this recursive relationship, an OnlineID attribute is present in the Console Relation.

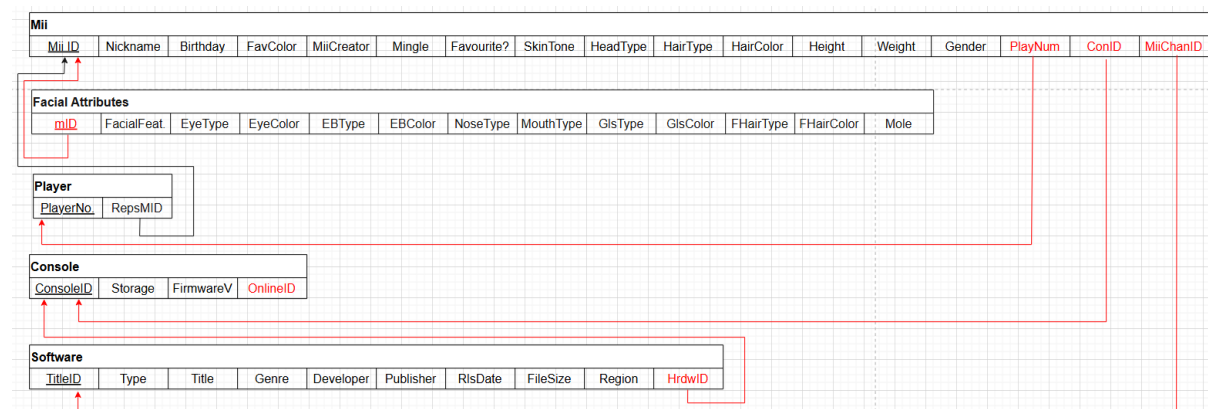


Figure 2d. Step 4 in mapping relational schema

Step 5: Mapping of Binary M:N Relationship Types

Many Miis can be used in various forms of software, whether it be a game or channel. To map this, a new relation used_in was created and included in the schema. This relation contains the primary keys of Mii and Software to use in its own composite primary key {SID,MID}.

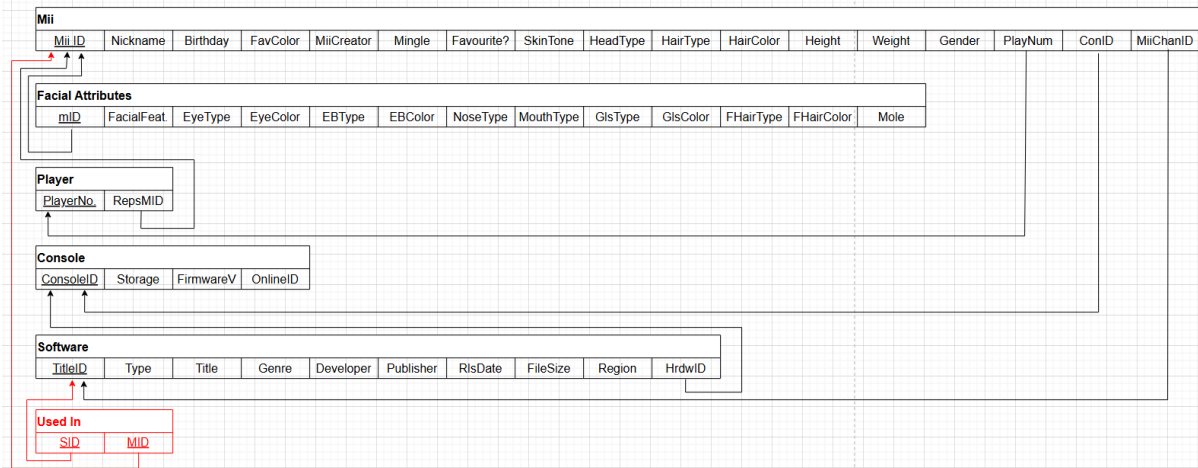


Figure 2e. Step 5 in mapping relational schema

Step 6: Mapping of Multivalued Attributes

No multivalued attributes are used in the ER diagram.

Step 7: Mapping of N-ary Relationship Types

No N-ary relationships are present in the ER diagram.

Final Relational Schema

By utilising this algorithm, the final relational schema was mapped:

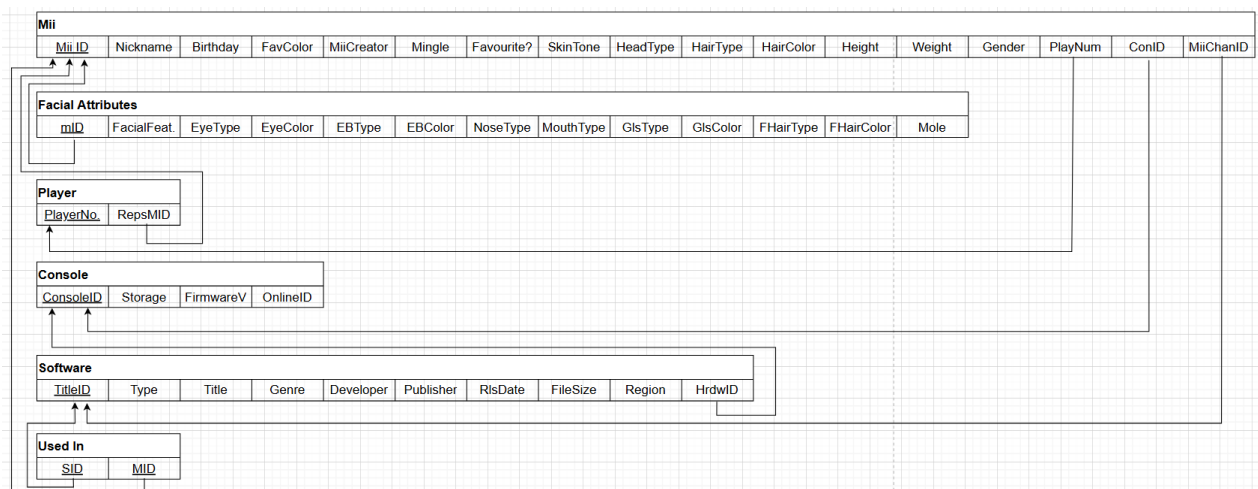


Figure 2f. Final relational schema

4. Explanation and SQL Code for Creating the database

➤ The “miisql” Database

Using the MySQL Command Client, the database was initially set up and given the name “miisql” using the following code:

```
DROP DATABASE IF EXISTS miisql;
CREATE DATABASE miisql;
USE miisql;
```

From there, MySQL Workbench was used to create each table and define their respective attributes. By using the ‘Forward Engineering’ option, the tables were converted to raw SQL code. Using this code, it can be seen directly when and how each table was created:

❖ Creating “mii” Table

```
DROP TABLE IF EXISTS `mii`;
CREATE TABLE `mii` (
  `MiiID` int NOT NULL COMMENT 'Identifier unique to every Mii',
  `Nickname` varchar(15) NOT NULL COMMENT 'Name of Mii',
  `Birthday` date DEFAULT NULL COMMENT '(Optional) NULL -> ??/??/????',
  `Favorite Colour` varchar(2) NOT NULL DEFAULT 'R' COMMENT 'Shirt colour(R-Red, O-Orange, Y-Yellow, LG-LightGreen, G-Green, B-Blue, LB-LightBlue, P-Pink, PR-Purple, BR-Brown, W-White ,BK-Black)',
  `Mii Creator` varchar(15) DEFAULT NULL COMMENT ' (Optional) Name/Alias of player who created current Mii',
  `Mingle` tinyint NOT NULL DEFAULT '0' COMMENT ' (Optional) Enable sharing mii creations with other consoles [via Online/Remote Transfer]',
  `Favorite?` tinyint NOT NULL DEFAULT '0' COMMENT ' (Optional) For Sorting Purposes [pants change from grey to dark red]',
  `Gender` VARCHAR(1) NOT NULL COMMENT 'M-Male, F-Female' ;
  `SkinTone` varchar(3) NOT NULL DEFAULT '1,1' COMMENT ' Format: (row),(column)',
  `HeadType` varchar(3) NOT NULL DEFAULT '1,1' COMMENT ' Format: (row),(column)',
  `HairType` varchar(5) DEFAULT '1.1.1' COMMENT ' Format: (page),(row),(column)\\n<3,4,1> = "Bald"(/null)',
  `Hair Colour` varchar(3) NOT NULL DEFAULT '1,2' COMMENT ' Format: (row),(column)',
  `Height` int NOT NULL DEFAULT '5' COMMENT ' Scale: 1(Shortest)-10(Tallest)',
  `Weight` int NOT NULL DEFAULT '5' COMMENT ' Scale: 1(Thinnest)-10(Largest)',
  `PlayNum` int NOT NULL COMMENT ' Foreign Key -> Player',
  `ConID` varchar(19) NOT NULL COMMENT ' Foreign Key -> Console',
  `MiiChanID` varchar(6) NOT NULL COMMENT ' Foreign Key -> Channel',
  PRIMARY KEY (`MiiID`),
  UNIQUE KEY `MiiID_UNIQUE` (`MiiID`),
  KEY `ConID_idx` (`ConID`),
  KEY `MiiChanID_idx` (`MiiChanID`),
  KEY `PlayNum_idx` (`PlayNum`),
  CONSTRAINT `ConID` FOREIGN KEY (`ConID`) REFERENCES `console` (`ConsoleID`),
  CONSTRAINT `MiiChanID` FOREIGN KEY (`MiiChanID`) REFERENCES `software` (`TitleID`),
  CONSTRAINT `PlayNum` FOREIGN KEY (`PlayNum`) REFERENCES `player` (`PlayerNo`)
)
```

Mii Constraints

- **Primary Key (MiiID):** Uniquely identifies each Mii record.
- **Unique Key (MiiID_UNIQUE):** Ensures that MiiID remains unique across all entries

- **Indexes (ConID_idx, MiiChanID_idx, PlayNum_idx):** These indexes improve the query performance when joining or searching the console, software, and player tables, respectively.
- **Foreign Keys:**
 - **ConID** references the ConsoleID in the console table, ensuring that each Mii is associated with a valid console.
 - **MiiChanID** references the TitleID in the software table to specify the Mii Channel.
 - **PlayNum** references PlayerNo in the player table, linking each Mii to a valid player.

❖ Creating “facial_attributes” Table

```
DROP TABLE IF EXISTS `facial_attributes`;
CREATE TABLE `facial_attributes` (
  `miID` int NOT NULL,
  `FacialFeatures` varchar(3) DEFAULT NULL COMMENT 'Freckles, Wrinkles, Makeup, etc.    Format: (row),(column)',
  `EyeType` varchar(5) NOT NULL DEFAULT '1,1,1' COMMENT '          Format: (page),(row),(column)',
  `EyeColour` varchar(3) NOT NULL DEFAULT '1,1' COMMENT '          Format: (row),(column)',
  `EyeBrowType` varchar(5) NOT NULL DEFAULT '1,1,1' COMMENT '          Format: (page),(row),(column)',
  `EyeBrowColour` varchar(3) NOT NULL DEFAULT '1,1' COMMENT '          Format: (row),(column)',
  `NoseType` varchar(3) NOT NULL DEFAULT '1,1' COMMENT '          Format: (row),(column)',
  `MouthType` varchar(5) NOT NULL DEFAULT '1,1,1' COMMENT '          Format: (page),(row),(column)',
  `GlassesType` varchar(3) DEFAULT NULL COMMENT '          Format: (row),(column)',
  `GlassesColour` varchar(2) NOT NULL DEFAULT 'BK' COMMENT 'Changes glasses colour    (BK-Black, BR-Brown, R-Red, B-Blue, Y-Yellow, W-White)',
  `FacialHairType` varchar(15) NOT NULL COMMENT '          Format: [Beard(B)](row),(column)
  [Mustache(m)](row),(column)',
  `FacialHairColour` varchar(3) NOT NULL DEFAULT '1,1' COMMENT '          Format: (row),(column)',
  `Mole` tinyint(1) NOT NULL DEFAULT '0',
  PRIMARY KEY (`miID`),
  KEY `mID_idx` (`miID`),
  CONSTRAINT `miID` FOREIGN KEY (`miID`) REFERENCES `mii` (`MiiID`)
)
```

facial_attributes Constraints

- **Primary Key(miID):** Ensures each record is uniquely identified by a corresponding Mii ID.
- **Indexes (mID_idx):** Optimises queries that search by miID, enhancing performance when accessing facial attributes linked to a specific Mii.
- **Foreign Key(miID):** References the MiiID in the Mii table, ensuring that each facial_attributes record corresponds to an existing Mii.

❖ Creating “player” Table

```
DROP TABLE IF EXISTS `player`;
CREATE TABLE `player` (
  `PlayerNo` int NOT NULL COMMENT 'Player Number of Individual during Mii creation/most recent use in software',
  `RepsMID` int NOT NULL,
```

```
PRIMARY KEY (`PlayerNo`),
KEY `RepsMID_idx` (`RepsMID`),
CONSTRAINT `RepsMID` FOREIGN KEY (`RepsMID`) REFERENCES `mii` (`MiiID`)
)
```

Player Constraints

- [1.] **Primary Key (PlayerNo):** Each player's number must be unique and serves as the main identifier for each record in this table.
- [2.] **Index (RepsMID_idx):** Optimises performance of queries that involve looking up records by RepsMID.
- [3.] **Foreign Key (RepsMID):** Establishes a relationship between the player table and the mii table, ensuring that every RepsMID value in the player table points to an existing Mii.

❖ Creating "console" Table

```
DROP TABLE IF EXISTS `console`;
CREATE TABLE `console` (
  `ConsoleID` varchar(19) NOT NULL COMMENT '          Format: "1234-5678-9012-3456"',
  `Storage(MB)` varchar(12) NOT NULL COMMENT 'Internal Storage for Channels/WiiWare  Format: "[TotChannelSize+
SystemFiles(250)]/Capacity(512) "',
  `FirmwareVersion` varchar(4) NOT NULL COMMENT '          Format: "1.2(region)" [Where region -> U(USA),
E(Europe), J(Japan)]',
  `OnlineID` varchar(15) NOT NULL COMMENT '          Format: "123 456 789 012"',
  PRIMARY KEY (`ConsoleID`)
)
```

Console Constraints

- **PRIMARY KEY (ConsoleID):** Enforces uniqueness in ConsoleID values, ensuring each console has a distinct identifier.

❖ Creating "software" Table

```
DROP TABLE IF EXISTS `software`;
CREATE TABLE `software` (
  `TitleID` varchar(6) NOT NULL DEFAULT '-',
  `Type` varchar(45) NOT NULL,
  `Title` varchar(45) NOT NULL,
  `Genre` varchar(15) NOT NULL,
  `Developer` varchar(45) NOT NULL,
  `Publisher` varchar(45) NOT NULL,
  `ReleaseDate` date NOT NULL,
  `FileSize(GB)` double NOT NULL COMMENT 'Size of data stored on disc',
  `Region` varchar(6) NOT NULL COMMENT '"NTSC-J" -> Japan/Asia "NTSC-U" -> Americas "NTSC-C" -> China "PAL" ->
Europe, New Zealand, Australia, Oceania, Middle East, India, South Africa',
  `HrdwID` varchar(19) NOT NULL COMMENT 'Format: "1234 5678 9012 3456"',
  PRIMARY KEY (`TitleID`),
  UNIQUE KEY `TitleID_UNIQUE` (`TitleID`),
  KEY `HrdwID_idx` (`HrdwID`),
  CONSTRAINT `HrdwID` FOREIGN KEY (`HrdwID`) REFERENCES `console` (`ConsoleID`)
```

Software Constraints

- [4.] **Primary Key (TitleID):** Every piece of Wii software, be it disc or channel, is distinguished by its own unique identifier known as a TitleID, making it an effective primary key for the software entity.
- [5.] **Index (HrdwID_idx):** Optimises performance of queries that involve looking up software by its associated console ID.
- [6.] **Foreign Key (HrdwID):** Establishes a relationship between the HrdwID column in the software table and the ConsoleID column in the console table to ensure that each software is associated with the console it is stored on/read from.

❖ Creating “used_in” Table

```
DROP TABLE IF EXISTS `used_in`;  
CREATE TABLE `used_in` (  
  `SID` varchar(6) NOT NULL,  
  `MID` int NOT NULL COMMENT 'ID of most recently used MII in software',  
  PRIMARY KEY (`SID`,`MID`),  
  KEY `MID_idx` (`MID`),  
  KEY `SID_idx` (`SID`),  
  CONSTRAINT `MID` FOREIGN KEY (`MID`) REFERENCES `mii` (`MiiID`),  
  CONSTRAINT `SID` FOREIGN KEY (`SID`) REFERENCES `software` (`TitleID`)  
)
```

used_in Constraints

- [7.] **Primary Key (SID, MID):** The primary key of the used_in table is a composite key, made up of both the SID and MID attributes. Ensures that each combination of a software and a Mii is unique in the table, preventing duplicate entries for the same Mii being used in the same software.
- [8.] **Indexes (SID_idx,MID_idx):** These indexes improve the query performance on their respective reference columns that are joined or searched.
- [9.] **Foreign Keys (SID, MID):** As they each refer to the primary keys of other tables, SID and MID are both considered as foreign keys in the used_in table.

5. Explanation and SQL Code for Populating the Tables

After creating each table, the 'Select Rows' tab in the MySQL Workbench GUI was utilised to populate it with data from a separately prepared Excel spreadsheet. Below is the code used to populate each table received via performing a 'Data export' through the MySQL Workbench GUI:

Populating mii Table:

```
LOCK TABLES `mii` WRITE;  
INSERT INTO `mii` (`MiiID`, `Nickname`, `Birthday`, `Favorite Colour`, `Mii Creator`, `Mingle`, `Favorite?`,  
`Gender`, `SkinTone`, `HeadType`, `HairType`, `Hair Colour`, `Height`, `Weight`, `PlayNum`, `ConID`, `MiiChanID`) VALUES  
(0, 'Matt', '2006-08-30', 'O', '"Me"', 0, 1, 'M', '2,2', '4,2', '3,4,1', '1,1', 5, 5, 1, '2351-4597-5015-9520', 'HACA'),  
(1, 'Hamburger', NULL, 'O', NULL, 1, 0, 'M', '1,3', '3,2', '3,4,1', '1,1', 1, 10, 2, '2351-4597-5015-9520', 'HACA'),  
(2, 'Lucía', '2006-08-27', 'LG', NULL, 0, 1, 'F', '1,1', '3,1', '4,3,2', '2,3', 5, 5, 3, '2351-4597-5015-9520', 'HACA'),
```

```
(3,'Peter G.',NULL,'W','\'"#1FamGuyFan\'"',1,0,'M','2,1','2,2','1,1,3','1,3',8,10,4,'5602-2894-8985-9937','HACA');
UNLOCK TABLES;
```

Populating facial_attributes Table:

```
LOCK TABLES `facial_attributes` WRITE;
INSERT INTO `facial_attributes` (`miID`, `FacialFeatures`, `EyeType`, `EyeColour`, `EyeBrowType`, `EyeBrowColour`,
`NoseType`, `MouthType`, `GlassesType`, `GlassesColour`, `FacialHairType`, `FacialHairColour`, `Mole`) VALUES
(0,'4,1','4,1,2','1,1','1,2,2','1,1','2,1','1,3,2',NULL,'1,1','[B]2,1 [M]2,2','1,1',0),
(1,'1,3','3,4,1','2,3','2,1,3','1,2','1,3','2,1,1',NULL,'1,1','[B]NULL [M]1,2','1,2',0),
(2,'2,1','2,1','1,3','1,2,1','1,1','1,1','1,2,1',NULL,'1,1','[B]NULL [M]NULL','1,1',0),
(3,NULL,'3,3,3','1,2','2,3,2','1,1','3,1','2,3,1','2,1','1,1','[B]NULL [M]1,2','2,4',1);
UNLOCK TABLES;
```

Populating player Table:

```
LOCK TABLES `player` WRITE;
INSERT INTO `player` (`PlayerNo`, `RepsMID`) VALUES
(1,0),
(2,1),
(3,2),
(4,3);
UNLOCK TABLES;
```

Populating console Table:

```
LOCK TABLES `console` WRITE;
INSERT INTO `console` (`ConsoleID`, `Storage(MB)`, `FirmwareVersion`, `OnlineID`) VALUES
('2351-4597-5015-9520','364.56/512','4.3E','209-591-407-786'),
--Peter.G Mii's console of origin (indicating they were received via online sharing)
('5602-2894-8985-9937','490.72/512','4.3U','484-827-161-976');
UNLOCK TABLES;
```

Populating software Table:

```
LOCK TABLES `software` WRITE;
INSERT INTO `software` (`TitleID`, `Type`, `Title`, `Genre`, `Developer`, `Publisher`, `ReleaseDate`,
`FileSize(GB)`, `Region`, `HrdwID`) VALUES
('HACA','Channel','Mii Channel','Utility','Nintendo','Nintendo','2006-12-08',0.0082,'PAL','2351-4597-5015-9520'),
('HAPP','Channel','Mii Contest Channel','Utility','Nintendo','Nintendo','2007-11-12',0.0218,'PAL','2351-4597-5015-9520'),
('RF8P69','Game(Disc)','Wii Sports','Sports','Nintendo EAD','Nintendo','2006-12-08',0.63,'PAL','2351-4597-5015-9520'),
('RMCP01','Game(Disc)','Mario Kart Wii','Racing','Nintendo EAD','Nintendo','2008-04-11',2.6,'PAL','2351-4597-5015-9520'),
('RODP01','Game(Disc)','Mario & Sonic at the Olympic Games','Sports','Sega Sports R&D','Sega, Nintendo','2007-11-23',1.89,'PAL','2351-4597-5015-9520'),
```

```
( 'RSPP01','Game(Disc)','WarioWare: Smooth Moves','Minigame','Nintendo SPD, Intelligent Systems','Nintendo','2007-01-12',3.27,'PAL','2351-4597-5015-9520'),
( 'RWSP8P','Game(Disc)','Wii Sports Resort','Sports','Nintendo EAD','Nintendo','2009-07-24',0.83,'PAL','2351-4597-5015-9520'),
( 'RZTP01','Game(Disc)','FIFA 08','Sports','EA Canada','EA Sports','2007-09-27',3.13,'PAL','2351-4597-5015-9520'),
( 'SNCP8P','Game(Disc)','Sonic Colours','Platformer','Sonic Team','Sega','2010-11-12',3.53,'PAL','2351-4597-5015-9520'),
( 'SUPP01','Game(Disc)','Wii Party','Party','NDCube, Nintendo SPD','Nintendo','2010-10-08',1.02,'PAL','2351-4597-5015-9520'),
( 'WDMP','Game(WiiWare)','Dr. Mario Online Rx','Puzzle','Arika','Nintendo','2008-05-20',0.0163,'PAL','2351-4597-5015-9520'),
( 'WM8P','Game(WiiWare)','Bomberman Blast','Action','Hudson Soft','Hudson Soft','2008-09-12',0.031,'PAL','2351-4597-5015-9520'),
( 'WUNP','Game(WiiWare)','Uno','Card','GameLoft','GameLoft','2009-11-05',0.0373,'PAL','2351-4597-5015-9520');
UNLOCK TABLES;
```

Populating used_in Table:

```
LOCK TABLES `used_in` WRITE;
INSERT INTO `used_in` (`SID`, `MID`) VALUES
( 'RMCP01',0),
( 'SNCP8P',0),
( 'WDMP',0),
( 'RWSP8P',1),
( 'RZTP01',1),
( 'WM8P',1),
( 'HAPP',2),
( 'RF8P69',2),
( 'RSPP01',2),
( 'HACA',3),
( 'RODP01',3),
( 'SUPP01',3),
( 'WUNP',3);
UNLOCK TABLES;
```

6. Explanation and SQL Code for Querying the database

The database could be queried using the MySQL Command Client. The code below shows how the database was queried to display a complete view of all entities and their data, retrieve data from the Mii table specifically, return all Miis that had white as their favourite colour, and retrieve all software data where a specific Mii was recently used.

```
-- Complete database view
SHOW DATABASES;
USE miisql;
SHOW TABLES;
```



```
DESCRIBE mii;  
SELECT * FROM mii;  
  
DESCRIBE facial_attributes;  
SELECT * FROM facial_attributes;  
  
DESCRIBE player;  
SELECT * FROM player;  
  
DESCRIBE console;  
SELECT * FROM console;  
  
DESCRIBE software;  
SELECT * FROM software;  
  
DESCRIBE used_in;  
SELECT * FROM used_in;
```

Output:

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| miisql      |
| mysql      |
| performance_schema |
| sakila     |
| sys       |
| world     |
+-----+
7 rows in set (0.05 sec)

mysql> USE miisql;
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_miisql |
+-----+
| console          |
| facial attributes |
| mii              |
| player          |
| software         |
| used in         |
+-----+
6 rows in set (0.01 sec)

mysql>
mysql>
mysql> DESCRIBE mii;
```

Field	Type	Null	Key	Default	Extra
MiID	int	NO	PRI	NULL	
Nickname	varchar(15)	NO		NULL	
Birthday	date	YES		NULL	
Favorite Color	varchar(2)	NO		R	
Mii Creator	varchar(15)	YES		NULL	

```

Mingle          tinyint      NO          0
Favorite?       tinyint      NO          0
Gender          varchar(1)   NO         NULL
SkinTone        varchar(3)   NO         1,1
HeadType        varchar(3)   NO         1,1
HairType        varchar(5)   YES        NULL
Hair Color      varchar(3)   NO         1,2
Height          int          NO          5
Weight          int          NO          5
PlayNum         int          NO         MUL    NULL
ConID           varchar(19)  NO         MUL    NULL
MiiChanID       varchar(6)   NO         MUL    NULL

```

```

17 rows in set (0.01 sec)

```

```

mysql> SELECT * FROM mii;

```

MiiID	Nickname	Birthday	Favorite Color	Mii Creator	Mingle	Favorite?	Gender	SkinTone	HeadType	HairType	Hair Color	Height	Weight
ght	PlayNum	ConID	MiiChanID										
0	Matt	2806-08-30	0	"Me"	0	1	M	2,2	4,2	3,4,1	1,1	5	
1	Hamburger	2351-4597-5015-9520	HACA	NULL	1	0	M	1,3	3,2	3,4,1	1,1	1	
2	Lucia	2806-08-27	LG	NULL	0	1	F	1,1	3,1	4,3,2	2,3	5	
3	Peter G.	2351-4597-5015-9520	HACA	"#1FamGuyFan"	1	0	M	2,1	2,2	1,1,3	1,3	8	
4		5692-2894-8985-9937	HACA										

```

4 rows in set (0.02 sec)

```

```
mysql> DESCRIBE facial_attributes;
```

Field	Type	Null	Key	Default	Extra
miID	int	NO	PRI	NULL	
FacialFeatures	varchar(3)	YES		NULL	
EyeType	varchar(5)	YES		NULL	
EyeColour	varchar(3)	NO		1,1	
EyeBrowType	varchar(5)	YES		NULL	
EyeBrowColor	varchar(3)	NO		1,2	
NoseType	varchar(3)	NO		1,1	
MouthType	varchar(5)	NO		1,1,1	
GlassesType	varchar(3)	YES		NULL	
GlassesColour	varchar(2)	NO		BK	
FacialHairType	varchar(15)	NO		NULL	
FacialHairColor	varchar(3)	NO		1,1	
Mole	tinyint(1)	NO		0	

```
13 rows in set (0.01 sec)

mysql> SELECT * FROM facial_attributes;
```

miID	FacialFeatures	EyeType	EyeColour	EyeBrowType	EyeBrowColor	NoseType	MouthType	GlassesType	GlassesColour	FacialHairType	FacialHairColor	Mole
0	4,1	0	4,1,2	1,1	1,2,2	1,1	2,1	1,3,2	NULL	BK	[B]2,1 [M]2,2	1,1
1	1,3	0	3,4,1	2,3	2,1,3	1,2	1,3	2,1,1	NULL	BK	[B]NULL [M]1,2	1,2
2	2,1	0	2,1	1,3	1,2,1	1,1	1,1	1,2,1	NULL	BK	[B]NULL [M]NULL	1,1
3	NULL	1	3,3,3	1,2	2,3,2	1,1	3,1	2,3,1	2,1	BK	[B]NULL [M]1,2	2,4

```
4 rows in set (0.00 sec)
```

Etc.

```
-- Query to retrieve all Mii entities
```

```
SELECT * FROM `mii`;
```

Output:

```
mysql> SELECT * FROM `mii`;
```

MiiID	Nickname	Birthday	Favorite Color	Mii Creator	Mingle	Favorite?	Gender	SkinTone	HeadType	HairType	Hair Color	Height	Weight
0	Matt	2006-08-30	0	"Me"	0	1	M	2,2	4,2	3,4,1	1,1	5	5
1	Hamburger	2351-4597-5015-9520	HACA	NULL	1	0	M	1,3	3,2	3,4,1	1,1	1	1
2	Lucia	2006-08-27	LG	NULL	0	1	F	1,1	3,1	4,3,2	2,3	5	5
3	Peter G.	2351-4597-5015-9520	HACA	"#1FamGuyFan"	1	0	M	2,1	2,2	1,1,3	1,3	8	8

```
4 rows in set (0.01 sec)
```

```
-- Query to retrieve all software entities where the genre is "Sports"
```

```
SELECT * FROM `software`
```

```
WHERE `Genre` = `Sports`;
```

Output:

```
mysql> SELECT * FROM software
-> WHERE `Genre` = 'Sports';
```

TitleID	Type	Title	Genre	Developer	Publisher	ReleaseDate	FileSize(GB)	Region	HrdwID
RF8P69	Game(Disc)	Wii Sports	Sports	Nintendo EAD	Nintendo	2006-12-08	0.63	PAL	2351-4597-50
RODP01	Game(Disc)	Mario & Sonic at the Olympic Games	Sports	Sega Sports R&D	Sega, Nintendo	2007-11-23	1.89	PAL	2351-4597-50
RWSP80	Game(Disc)	Wii Sports Resort	Sports	Nintendo EAD	Nintendo	2009-07-24	0.83	PAL	2351-4597-50
RZTP01	Game(Disc)	FIFA 08	Sports	EA Canada	EA Sports	2007-09-27	3.13	PAL	2351-4597-50

```
4 rows in set (0.00 sec)
```

```
-- Query to retrieve all Title IDs from software where a Mii has been recently used in
```

```
SELECT * FROM `used_in`
```

```
WHERE `MID` = '3';
```

Output:

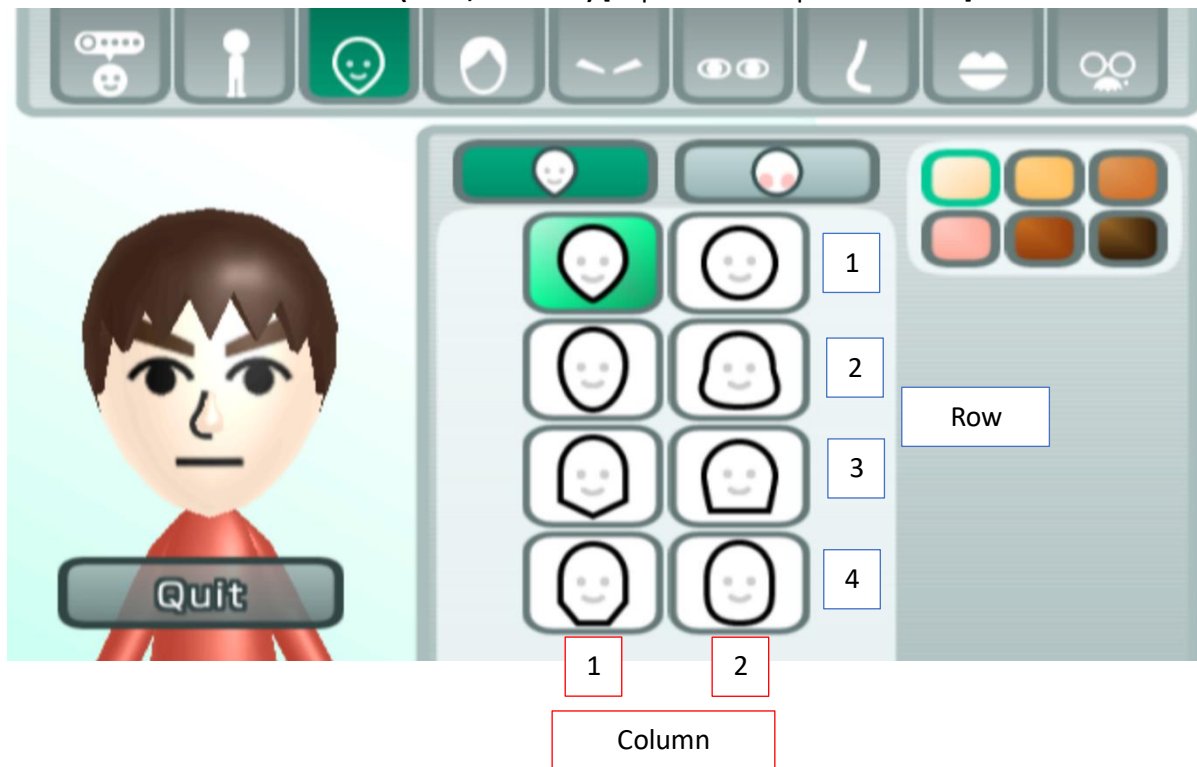
```
mysql> SELECT * FROM `used_in`
-> WHERE `MID` = '3';
```

SID	MID
HACA	3
RODP01	3
SUPP01	3
WUNP	3

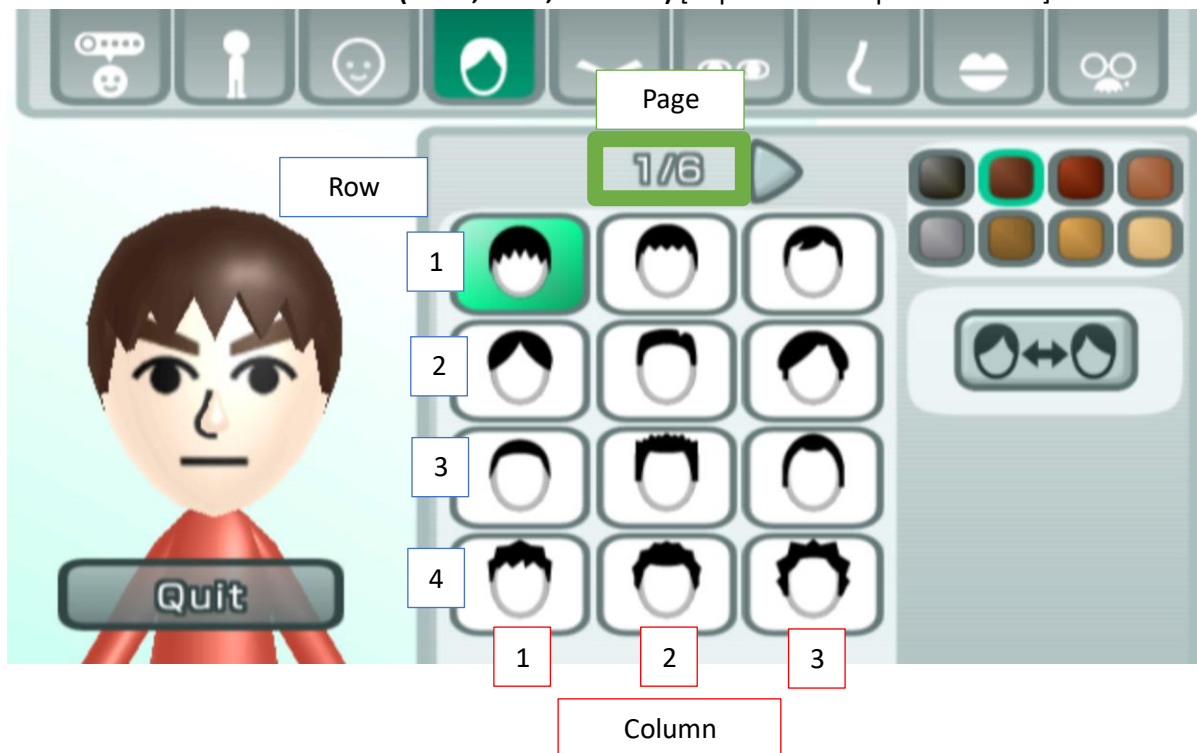
```
4 rows in set (0.00 sec)
```

APPENDICE

A1. Visual Guide of Grid Format (ROW,COLUMN) [Captured in Dolphin Emulator]



A2. Visual Guide of Grid Format (PAGE, ROW,COLUMN) [Captured in Dolphin Emulator]



A3. SQL Source Code

A text file containing the complete source code of the database is included with the submission of this assignment.

References

- [1.] FAQ/help - What is unused Mii data?, Mii Library. Available at:
<https://sites.google.com/view/miilibrary/contactfaqother/FAQ#h.bs18r4ccl77b> (Accessed: 02 November 2024).

- [2.] *List of video games featuring Miis* (2024) *Wikipedia*. Available at:
https://en.wikipedia.org/wiki/List_of_video_games_featuring_Miis (Accessed: 07 November 2024).